

AUTOMATED WEB VULNERABILITY SCANNER WITH REMEDIATION ENGINE

A Project Report submitted in partial fulfillment of the requirements for the award of

**BACHELOR OF SCIENCE
IN
COMPUTER SCIENCE WITH CYBER SECURITY**

of Bharathiar University, Coimbatore-46.

by

ARYA GIRISHKUMAR

Reg. No:2328N0036

Under the guidance of

Dr. SAJITHRA N , MSc, M.Phil, PhD

Assistant Professor

AI and Cyber Security Programme

Department of Computer Science



SREE NARAYANA GURU COLLEGE

K.G.CHAVADI, COIMBATORE-641105.

(An Autonomous College, Affiliated to Bharathiar University and

Accredited with "A" grade by NAAC &

An ISO 9001:2015 Certified Institution)

March - 2026

AUTOMATED WEB VULNERABILITY SCANNER WITH REMEDiation ENGINE

Bonafide Work Done by

ARYA GIRISHKUMAR

Reg. No:2328N0036

Under the guidance of

Dr. SAJITHRA N , MSc, M.Phil, PhD

Assistant Professor

AI and Cyber Security Programme

Department of Computer Science

Project Report submitted in partial fulfillment of the requirements for the award of

BACHELOR OF SCIENCE

IN

COMPUTER SCIENCE WITH CYBER SECURITY

of Bharathiar University, Coimbatore-105.



Signature of the Guide

Signature of the Programme Coordinator

Signature of the HOD

Submitted for the Viva Voce Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

MARCH 2026

DECLARATION

I, ARYA GIRISHKUMAR here by declaring that the project report titled “**AUTOMATED WEB VULNERABILITY SCANNER WITH REMEDIATION ENGINE**” done as a partial fulfillment of the requirement for the degree of **BACHELOR OF SCIENCE IN COMPUTER SCIENCE WITH CYBER SECURITY** is an independent project during the project duration of one period of study at **SREE NARAYANA GURU COLLEGE**, K.G. Chavadi, Coimbatore, under the guidance of **Dr. SAJITHRA N, MSc , M.Phil , PhD**, Assistant Professor, AI and Cyber Security programme of the Department of Computer Science.

Signature of the Guide

Signature of the Student

ARYA GIRISHKUMAR

CONTENTS

	Page No.
ACKNOWLEDGEMENT	(i)
SYNOPSIS	(ii)
1. INTRODUCTION	01
1.1 ORGANIZATION PROFILE	01
1.2 SYSTEM SPECIFICATION	02
1.2.1 HARDWARE CONFIGURATION	02
1.2.2 SOFTWARE SPECIFICATION	02
2. SYSTEM STUDY	03
2.1 EXISTING SYSTEM	03
2.1.1 DRAWBACKS	03
2.2 PROPOSED SYSTEM	04
2.2.1 FEATURES	04
3. SYSTEM DESIGN AND DEVELOPMENT	05
3.1 FILE DESIGN	05
3.2 INPUT DESIGN	05
3.3 OUTPUT DESIGN	06
3.4 DATABASE DESIGN	06
3.5 SYSTEM DEVELOPMENT	07
3.5.1 DESCRIPTION OF MODULES	08
4. TESTING AND IMPLEMENTATION	09
5. CONCLUSION	11
6. BIBLIOGRAPHY	12
7. APPENDICES	13
A. DATA FLOW DIAGRAM	13
B. TABLE STRUCTURE	15
C. SAMPLE CODING	17
D. SAMPLE INPUT	23
E. SAMPLE OUTPUT	24

ACKNOWLEDGEMENT

First and foremost,I express my sincere thanks to our beloved and respected **Dr. D Kalpana PhD**, Principal, Sree Narayana Guru College , K.G Chavadi ,Coimbatore.

I would like to take this opportunity to express my gratitude to **Dr. R Priya MCA, M.Phil, PhD**, Professor & Head, Department of Computer Science, Sree Narayana Guru College, who permitted me to do this project work.

I would like to take this opportunity to express my gratitude to **Ms. Ramya S, MCA, M.Phil, NET, (PhD)**, Assistant Professor and Programme Coordinator, AI and Cyber Security programme ,Department of Computer Science, Sree Narayana Guru College, for her ideas that helped me to complete the work.

I would like to this opportunity to express my sincere thanks to my project guide **Dr. Sajithra N , MSc, M.Phil, PhD**, Assistant Professor,AI and Cyber Security programme, Department of Computer science, Sree Narayana Guru College, for her valuable guidance, encouragement and helping me to do the project work.

I would like to express my heartfelt gratitude to all the Staff members of our department, Sree Narayana Guru College, K.G Chavadi, Coimbatore.

I would like to express my special thanks to my family and my friends who gave their assistance at each and every stage of this project.

SYNOPSIS

The rapid growth of web applications has increased the risk of cyber security threats and vulnerabilities. Many websites suffer from security issues such as SQL Injection, Cross-Site Scripting (XSS), and other common web vulnerabilities. Identifying these vulnerabilities manually is time-consuming and requires strong technical expertise. Therefore, an automated system is required to detect and address such security issues efficiently. The project “VulnQuest Pro – Automated Web Vulnerability Scanner with Remediation Engine” is designed to automatically scan web applications and identify potential security vulnerabilities. The system analyzes website inputs, URLs, and responses to detect possible threats. Once vulnerabilities are detected, the system provides remediation suggestions to help developers fix the issues. The proposed system improves the security of web applications by automating the vulnerability detection process. It helps developers and administrators identify risks early and apply appropriate security measures. The system is built using modern programming tools and security libraries to ensure accurate detection and efficient performance. This project aims to enhance web security by providing an easy-to-use platform that detects vulnerabilities and suggests remediation techniques, helping organizations maintain safer and more secure web applications.

1.INTRODUCTION

With the rapid growth of internet technology, web applications have become an essential part of modern businesses, organizations, and everyday activities such as online shopping, banking, education, and communication. As the number of web applications increases, the risk of cyber attacks and security vulnerabilities also grows. Many websites are vulnerable to threats like SQL Injection, Cross-Site Scripting (XSS), and other security weaknesses that can lead to data breaches, unauthorized access, and system damage. Identifying these vulnerabilities manually can be time-consuming and requires technical expertise. To overcome this challenge, VulnQuest Pro – Automated Web Vulnerability Scanner with Remediation Engine is developed as an automated solution to detect common security vulnerabilities in web applications. The system scans websites, identifies potential security risks, and generates detailed reports about the detected vulnerabilities. It also provides remediation suggestions to help developers understand and fix the issues effectively. By automating the vulnerability detection process and offering guidance for remediation, this project helps improve web application security and reduces the chances of cyber attacks.

1.1. ORGANIZATION PROFILE

Sree Narayana Guru College

With a view of imparting quality education for the aspiring youth of Coimbatore, Sree Narayana Guru Educational Trust members came forward with a plan to start a college for Applied Sciences and Management Studies. Sree Narayana Guru College (SNGC) came into existence in 1994, affiliated to Bharathiar University, accredited with “A” grade by NAAC (3rd cycle) and approved by Govt. of Tamil Nadu.

Sree Narayana Guru College strives to emerge as a premiere institute of international standards promoting excellence and equity in higher education. To mould the students of the institution through a socially committed, intellectually , value based and culture driven paradigms of learning using the state-of-the-art educational technologies.

The institution helps to enhance the learning capacity and knowledge of the students by imparting quality education. This makes the students think critically, objectively, creatively and to be a life -long learners and productive citizens. Through education, institutions aim to mold student’s character and develop a value system so as to manifest oneness among students of diverse socio-cultural and economic backgrounds.

1.2 SYSTEM SPECIFICATION

The system specification defines the hardware and software requirements needed to develop and run The AI Resume Scanning and Job Matching System. The application is a web-based system that uses Machine Learning and Natural Language Processing to analyze resumes, predict suitable job roles, recommend career domains, identify skill gaps, and generate improved resumes.

1.2.1 HARDWARE CONFIGURATION

Minimum Hardware Requirements:

- Processor : Intel Core i5 Processor
- RAM : 8 GB
- Storage : 512 GB SSD
- Display : 15.6 inch Full HD Display
- Input Devices : Keyboard and Touchpad
- Internet Connection : Wi-Fi Enabled

Recommended Hardware Requirements:

- Processor: Intel Core i5 or above
- RAM: 8 GB or higher
- Storage: SSD for better performance

1.2.2 SOFTWARE SPECIFICATION

- Operating System : Windows 11
- Programming Language : Python
- IDE / Code Editor : Visual Studio Code
- Web Framework : Flask
- Web Browser : Google Chrome
- Libraries / Tools : Requests (Python library), Beautiful soup.

2.SYSTEM STUDY

System study is the process of analyzing the existing system to understand its functionality, limitations, and requirements. It helps in identifying the problems in the current system and determining the need for a new or improved system. Through system study, the requirements of the proposed system are clearly understood and suitable solutions are developed. In this project, the system study focuses on analyzing the security issues in existing web applications and identifying the need for an automated vulnerability scanning system to improve web security and detect threats efficiently.

2.1 EXISTING SYSTEM

In the existing system, web application security testing is mainly done using manual methods or basic security tools. Developers or security analysts check websites for vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS). This process requires technical knowledge and takes more time to detect security issues.

Limitations of Existing System:

- Manual testing is time-consuming
- Requires strong technical knowledge
- Limited automation in vulnerability detection
- No proper guidance to fix detected vulnerabilities
- Less efficient for large web applications

2.1.1 DRAWBACKS

- Time Consuming – Manual vulnerability testing takes a lot of time.
- Requires Technical Expertise – Security testing needs skilled professionals.
- Limited Automation – Many existing methods do not provide fully automated scanning.
- Incomplete Detection – Some vulnerabilities may not be identified properly.
- No Remediation Support – Existing systems often do not provide clear solutions to fix the detected issues.

2.2 PROPOSED SYSTEM

The proposed system, VulnQuest Pro – Automated Web Vulnerability Scanner with Remediation Engine, is designed to automatically detect security vulnerabilities in web applications. The system scans websites to identify common threats such as SQL Injection and Cross-Site Scripting (XSS). It analyzes the web pages and identifies possible weaknesses that may affect the security of the web application.

The system generates detailed reports of the detected vulnerabilities and also provides remediation suggestions to help developers fix the identified issues. This automated approach reduces manual effort and improves the speed of vulnerability detection.

The proposed system also provides a user-friendly interface that allows users to easily enter the website URL and start the scanning process. This makes the system easy to use even for users with basic technical knowledge.

Additionally, the system maintains scan results and reports that can be used for future reference. By automating the vulnerability detection process and providing useful recommendations, the system helps improve the overall security of web applications and reduces the risk of cyber attacks.

2.2.1 FEATURES

- Automated Vulnerability Scanning – Automatically scans websites to detect security vulnerabilities.
- Detection of Common Attacks – Identifies threats such as SQL Injection and Cross-Site Scripting (XSS).
- Detailed Report Generation – Generates reports of detected vulnerabilities.
- Remediation Suggestions – Provides solutions to fix the identified issues.
- User-Friendly Interface – Simple interface for easy usage.
- Improved Web Security – Helps developers enhance the security of their web applications.

3. SYSTEM DESIGN AND DEVELOPMENT

System design and development is the phase where the structure and working of the proposed system are planned and implemented. In this stage, the system architecture, modules, and data flow are designed to ensure efficient performance.

3.1 FILE DESIGN

File design refers to the way files are organized and maintained in the system to store and manage data effectively. Proper file design helps in easy storage, retrieval, and management of information within the system.

In this project, different files are used to store user input, scanning results, vulnerability details, and generated reports. These files help the system maintain records of the scanning process and ensure that the detected vulnerabilities and related information are stored properly for future reference.

Files used in the system:

- Input File – Stores the website URL entered by the user for vulnerability scanning.
- Scan Data File – Stores the data collected during the scanning process.
- Vulnerability Result File – Contains details of detected vulnerabilities in the web application.
- Report File – Stores the generated vulnerability scan reports for the user.
- Log File – Maintains records of system activities and previous scanning results.
- Configuration File – Stores system settings and scanning parameters used by the application.
- User Data File – Stores user-related information required by the system.
- Temporary File – Used to store temporary data during the scanning process.

3.2 INPUT DESIGN

Input design defines how data is entered into the system for processing. A well-designed input system ensures accuracy, reduces errors, and makes the application easy to use.

- Website URL Input – The user enters the target website URL for vulnerability scanning.
- Input Validation – The system verifies whether the entered URL is valid before processing.
- User-Friendly Interface – The input interface is simple and easy for users to operate.
- Error Handling – The system displays error messages for invalid or incorrect inputs.
- Scan Request – The user can start the vulnerability scanning process after entering the required input.

3.3 OUTPUT DESIGN

Output design refers to the way the processed information is presented to the user. A good output design ensures that the results are clear, accurate, and easy to understand.

- Vulnerability Report – Displays the detected security vulnerabilities in the website.
- Detailed Scan Results – Shows information about each identified threat.
- Remediation Suggestions – Provides solutions to fix the detected vulnerabilities.
- User-Friendly Format – Results are presented in a clear and simple format.
- Report Storage – Generated reports can be saved for future reference.
- Error Notifications – Displays error messages if the scanning process fails.
- Summary Output – Provides a summary of total vulnerabilities detected.
- Downloadable Report – Allows users to download the vulnerability report if required.

3.4 DATABASE DESIGN

Database design refers to the process of organizing and structuring data in a database so that it can be stored, managed, and retrieved efficiently. A well-designed database helps maintain data

accuracy, reduces redundancy, and improves the overall performance of the system. It also ensures that the system can handle large amounts of data in an organized manner.

In the VulnQuest Pro – Automated Web Vulnerability Scanner with Remediation Engine, the database plays an important role in storing and managing information related to users, scanned websites, detected vulnerabilities, and generated reports. The database keeps a record of the scanning activities so that users can review previous scan results and analyze security issues in web applications.

The database is structured using different tables that store specific types of data. These tables are connected to ensure proper data management and easy retrieval of information when required. This structured design helps the system generate reports efficiently and maintain accurate records of vulnerabilities detected during the scanning process.

Main tables used in the database include:

- User Table – Stores user details such as username, email, and login credentials.
- Website Table – Stores information about the websites submitted for scanning.
- Scan Result Table – Contains details about the results of each vulnerability scan.
- Vulnerability Table – Stores information about different types of detected vulnerabilities.
- Report Table – Stores the generated scan reports for user reference.
- Log Table – Maintains records of scanning activities and system operations.

3.5 SYSTEM DEVELOPMENT

System development is the process of building and implementing the proposed system using appropriate technologies and tools. It involves coding, integrating modules, and testing the system to ensure proper functionality. The development process focuses on creating a reliable and efficient application for vulnerability detection.

In this project, the system is developed using Python and supporting libraries to perform web vulnerability scanning. Different modules such as input handling, vulnerability detection, and report generation are implemented to perform specific tasks. These modules work together to scan websites, detect security issues, and generate reports.

- Programming Language – The system is developed using Python.
- Development Environment – Visual Studio Code is used for coding and development.
- Scanning Module – Developed to detect vulnerabilities in web applications.
- Report Generation Module – Generates reports of detected vulnerabilities.
- User Interface – Designed to allow users to easily enter website URLs and start scanning.
- Testing – The system is tested to ensure accurate vulnerability detection and proper performance.

3.5.1 DESCRIPTION OF MODULES

The system is divided into different modules to perform specific tasks efficiently. Each module performs a particular function and works together with other modules to complete the vulnerability scanning process. This modular structure improves system performance and makes the application easier to maintain and update.

- User Input Module – This module allows the user to enter the website URL that needs to be scanned for vulnerabilities. It collects the required input from the user and sends it to the scanning module for further processing.
- Scanning Module – This module performs the main function of scanning the website. It accesses the web pages and analyzes them to find possible security weaknesses in the application.
- Vulnerability Detection Module – This module analyzes the scanned data and identifies common security threats such as SQL Injection and Cross-Site Scripting (XSS). It helps in detecting possible vulnerabilities that may affect the security of the web application.
- Report Generation Module – This module generates detailed reports based on the detected vulnerabilities. The report provides information about the type of vulnerability and where it occurs in the web application.

4. TESTING AND IMPLEMENTATION

4.1 IMPLEMENTATION

Implementation is the stage where the designed system is converted into a working application. In this phase, the system modules are developed, integrated, and executed using suitable programming tools and technologies. It ensures that the system functions according to the design and requirements.

In this project, the implementation is carried out using Python and related libraries for web vulnerability scanning. The system accepts the website URL as input, performs scanning to detect vulnerabilities, and generates reports with remediation suggestions. Each module such as input handling, vulnerability detection, and report generation is implemented and tested to ensure accurate and efficient system performance.

4.2 TESTING

Testing is the process of evaluating the system to ensure that it works correctly and meets the specified requirements. It helps in identifying errors, bugs, and issues in the system before it is used by users. Proper testing improves the reliability and performance of the application.

Types of Testing Performed

- Unit Testing – Each module of the system is tested individually to ensure it works correctly.
- Integration Testing – Different modules are tested together to check whether they work properly when combined.
- System Testing – The complete system is tested to verify that all functions operate as expected.
- Functional Testing – Tests whether the system performs its intended functions correctly.
- User Acceptance Testing – Ensures the system meets user requirements and is ready for use

4.3 TEST CASES

Test cases are used to verify whether the system functions correctly under different conditions. They help ensure that the application processes inputs properly and produces the expected outputs. Test cases are important for identifying errors and improving the reliability of the system. In this project, test cases are designed to check different operations of the system such as entering the website URL, starting the vulnerability scanning process, and generating the scan report. These tests help confirm that each function of the system works according to the specified requirements.

By performing various test cases, the system's accuracy and performance can be evaluated. This ensures that the application correctly detects vulnerabilities and provides reliable results to the users.

Test Case	Input	Expected Output
Check valid website URL	Valid URL entered	System starts vulnerability scanning
Check invalid website URL	Invalid URL entered	System displays error message
Start scanning process	User clicks scan button	System performs vulnerability scan
Generate scan report	Scan completed	System generates vulnerability report
View report	User requests report	System displays scan results

5. CONCLUSION

The VulnQuest Pro – Automated Web Vulnerability Scanner with Remediation Engine has been successfully developed to address the increasing need for web application security. With the growing reliance on websites for business, education, and communication, ensuring that web applications are protected from cyber threats has become critical. This system automates the detection of common vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and other security weaknesses, which are often exploited by attackers to gain unauthorized access or steal sensitive data.

The system not only identifies potential vulnerabilities but also generates detailed reports and provides remediation suggestions to help developers fix the issues efficiently. This reduces the time and effort required for manual testing and helps in maintaining a secure environment for web applications. The user-friendly interface ensures that even users with minimal technical knowledge can operate the system effectively, making it accessible to a wider range of developers and security professionals.

By integrating automated scanning, accurate vulnerability detection, and actionable remediation guidance, VulnQuest Pro significantly enhances web security. It provides a reliable, efficient, and practical solution for maintaining the safety of web applications, reducing the risk of cyber attacks, and supporting developers in building robust and secure websites.

6. BIBLIOGRAPHY

1. Stuttard, D., & Pinto, M. (2011). The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws. Wiley.
2. OWASP Foundation. (2025). OWASP Top Ten Web Application Security Risks. Available at: <https://owasp.org>
3. Joshi, R. (2020). Python Web Security Projects. Packt Publishing.
4. Chapple, M., & Seidl, M. (2018). CompTIA Security+ Guide to Network Security Fundamentals. Cengage Learning.
5. Hoffman, P. (2019). Practical Web Application Security: Securing Your Websites. Apress.
6. Official Python Documentation. (2025). Python.org. Available at: <https://docs.python.org/3/>
7. Flask Documentation. (2025). Flask Web Framework. Available at: <https://flask.palletsprojects.com/>

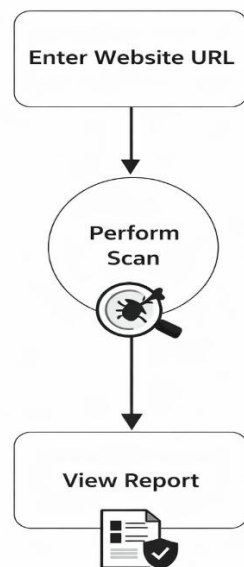
7. APPENDICES

A. DATA FLOW DIAGRAM (DFD)

Data Flow Diagram (DFD)

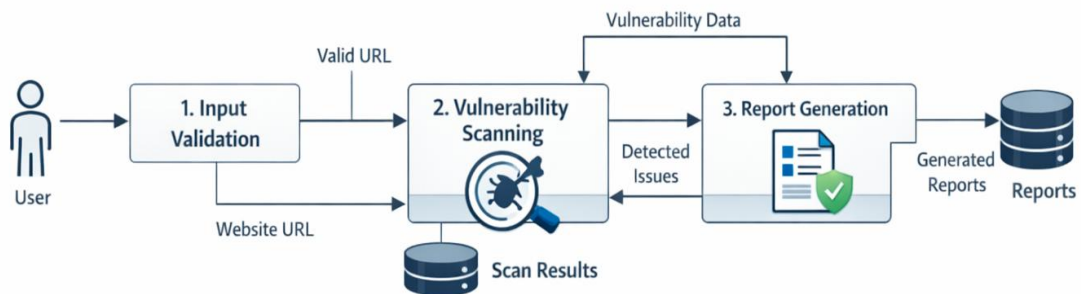
The data flow of the system begins with the user uploading a resume in PDF format. The Resume Parser extracts the text from the file and sends it to the analysis modules. The ATS Score module evaluates resume quality, and the Machine Learning model predicts suitable job roles. The system then maps the roles to career domains and identifies missing skills. Finally, the Resume Improver generates an enhanced resume, which can be downloaded as a PDF by the user.

DFD LEVEL 0



DFD LEVEL 1 (Detailed Diagram)

VulnQuest Pro – Automated Web Vulnerability Scanner with Remediation Engine



Data Flow Diagram (DFD)

B. TABLE STRUCTURE

The system uses structured tables to manage the data flow for web vulnerability scanning and analysis. The Scan Input Dataset table stores the list of target websites along with metadata such as scan date and type, which helps the system schedule and perform automated scans. The Scan Results table records all detected vulnerabilities, including their severity and descriptions, providing a clear overview of risks identified in each web application. Finally, the Remediation Status table tracks the actions taken to fix or mitigate these vulnerabilities, helping users monitor the progress of remediation and maintain the security posture of the scanned websites. Together, these tables provide a complete framework for organizing, analyzing, and managing web security data efficiently.

Table 1: Scan Input Dataset

This table contains the list of websites that will be scanned by the system. Each URL is stored with metadata such as scan date and type. The system uses this information to schedule and perform automated scans efficiently.

Field Name	Data Type	Description
url	VARCHAR(255)	Target website or web page URL
scan_date	DATE	Date when the scan is scheduled
scan_type	VARCHAR(50)	Type of scan (Full Scan, SQLi, XSS, etc.)

Table 2 : Scan Results

This table stores the results generated after scanning each website. It includes detected vulnerabilities, their severity, and a short description. This helps the user quickly understand the risks identified by the system.

Field Name	Data Type	Description
scan_id	INT (PK)	Unique ID for each scan
url	VARCHAR(255)	Target website scanned
vulnerability_name	VARCHAR(100)	Name of detected vulnerability
severity	VARCHAR(50)	Risk level (High / Medium / Low)
description	TEXT	Short description of the vulnerability

Table 3: Remediation Status

This table tracks the remediation actions taken for each detected vulnerability. It helps users and developers keep track of which vulnerabilities are fixed, pending, or not applicable

Field Name	Data Type	Description
scan_id	INT (FK)	Reference to the scan result
vulnerability_name	VARCHAR(100)	Name of detected vulnerability
remediation_status	VARCHAR(50)	Pending / Fixed / Not Applicable
remarks	TEXT	Additional notes or instructions

C. SAMPLE CODING

1. app.py

```
from flask import Flask, render_template, request, redirect, url_for, send_file
import sqlite3

from scanner import scan_target, export_csv, export_pdf

app = Flask(__name__)

DB = "database.db"

# Initialize Database

def init_db():

    conn = sqlite3.connect(DB)

    c = conn.cursor()

    c.execute("""

    CREATE TABLE IF NOT EXISTS scans (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    target TEXT,

    url TEXT,

    vulnerability TEXT,

    severity TEXT,

    description TEXT,

    solution TEXT

    )
```

```

"""
conn.commit()

conn.close()

init_db()

# Home Page

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        target = request.form["target"]
        results = scan_target(target)

    conn = sqlite3.connect(DB)
    c = conn.cursor()

    c.execute("DELETE FROM scans") # clear old scan

    for r in results:
        c.execute("""INSERT INTO scans
(target, url, vulnerability, severity, description, solution)
VALUES (?, ?, ?, ?, ?, ?)""",
(target, r["url"], r["vuln"], r["severity"], r["desc"], r["solution"])
)

    conn.commit()

    conn.close()

```



```

return redirect(url_for("dashboard"))

return render_template("index.html")


# Dashboard Page

@app.route("/dashboard")

def dashboard():

    conn = sqlite3.connect(DB)

    c = conn.cursor()

    c.execute("SELECT * FROM scans")

    rows = c.fetchall()

    conn.close()

    return render_template("dashboard.html", rows=rows)


# Download CSV

@app.route("/download/csv")

def download_csv():

    return send_file(export_csv(), as_attachment=True)


# Download PDF

@app.route("/download/pdf")

def download_pdf():

    return send_file(export_pdf(), as_attachment=True)


# CLI Support

```

```

if __name__ == "__main__":

import sys

if len(sys.argv) > 1:

target = sys.argv[1]

results = scan_target(target)

for r in results:

print(f"[{r['severity']}] [{r['vuln']} -> {r['url']}")

else:

app.run(debug=True)

```

2. scanner.py

```

import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin
import csv
from fpdf import FPDF

# Crawl the website
def crawl_site(target):
    urls = set()
    try:
        r = requests.get(target)
        soup = BeautifulSoup(r.text, "html.parser")
        for link in soup.find_all("a"):
            href = link.get("href")
            if href:
                full_url = urljoin(target, href)
                urls.add(full_url)
    except:
        pass
    return list(urls)

# Check SQL Injection
def check_sql_injection(url):
    payload = "' OR '1'='1"
    try:

```

```

        test_url = url + "?id=" + payload
        r = requests.get(test_url)
        if "sql" in r.text.lower() or "syntax" in r.text.lower():
            return True
    except:
        pass
    return False

# Check XSS
def check_xss(url):
    payload = "<script>alert(1)</script>"
    try:
        test_url = url + "?q=" + payload
        r = requests.get(test_url)
        if payload in r.text:
            return True
    except:
        pass
    return False

# Main Scanner
def scan_target(target):
    results = []
    urls = crawl_site(target)
    for url in urls:
        if check_sql_injection(url):
            results.append({
                "url": url,
                "vuln": "SQL Injection",
                "severity": "High",
                "desc": "SQL queries can be manipulated by user input",
                "solution": "Use prepared statements and input validation"
            })
        if check_xss(url):
            results.append({
                "url": url,
                "vuln": "Cross Site Scripting (XSS)",
                "severity": "Medium",
                "desc": "Malicious scripts can be injected",
                "solution": "Escape user input and use Content Security Policy"
            })
    return results

```

```

# Export CSV
def export_csv():
    filename = "report.csv"
    with open(filename, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["URL", "Vulnerability", "Severity", "Description", "Solution"])
    from app import DB
    import sqlite3
    conn = sqlite3.connect(DB)
    c = conn.cursor()
    c.execute("SELECT url, vulnerability, severity, description, solution FROM scans")
    for row in c.fetchall():
        writer.writerow(row)
    conn.close()
    return filename

# Export PDF
def export_pdf():
    filename = "report.pdf"
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Arial", size=10)
    from app import DB
    import sqlite3
    conn = sqlite3.connect(DB)
    c = conn.cursor()
    c.execute("SELECT url, vulnerability, severity, description, solution FROM scans")
    for row in c.fetchall():
        pdf.multi_cell(0, 8, f"URL: {row[0]}")
        pdf.multi_cell(0, 8, f"Vulnerability: {row[1]}")
        pdf.multi_cell(0, 8, f"Severity: {row[2]}")
        pdf.multi_cell(0, 8, f"Description: {row[3]}")
        pdf.multi_cell(0, 8, f"Solution: {row[4]}")
        pdf.ln()
    conn.close()
    pdf.output(filename)
    return filename

```

3.HTML Templates

```
<!DOCTYPE html>

<html>

<head>

    <title>VulnQuest Pro - Scan Website</title>

</head>

<body>

    <h1>VulnQuest Pro</h1>

    <form method="POST">

        <input type="text" name="target" placeholder="Enter website URL" required>

        <button type="submit">Scan</button>

    </form>

</body>

</html>
```

D. SAMPLE INPUT

In VulnQuest Pro, the user provides a website URL as input to start the scanning process. This can be done either through the web interface or via the command-line interface (CLI). For example, a test site such as <https://testphp.vulnweb.com> can be entered in the input form on the homepage. Once the URL is submitted, the system crawls the website, checks each page for vulnerabilities like SQL Injection and Cross-Site Scripting (XSS), and stores the results in the database. The detected vulnerabilities are then displayed on the dashboard, and the user can download the findings in CSV or PDF format. Using these sample inputs allows safe testing of the tool's functionality without targeting live websites. Example

Sample Input:

← → ↻ 📄 127.0.0.1:5000

VulnQuest Pro

Enter a website you own or have permission to test

E. SAMPLE OUTPUT

After scanning a website, VulnQuest Pro displays the detected vulnerabilities on the dashboard in a structured table format. Each entry shows the URL of the affected page, the type of vulnerability (e.g., SQL Injection or Cross-Site Scripting), its severity level (High, Medium, or Low), a brief description of the issue, and the recommended solution to fix it. For example, the login page of <https://testphp.vulnweb.com> might be flagged for a high-severity SQL Injection vulnerability, while the search page could show a medium-severity XSS issue. The dashboard also provides options to download the results as a CSV or PDF file, allowing users to save and share the findings. This output helps users understand security weaknesses and take corrective actions.

user and pin it to your taskbar

Set as default

Automated Web Vulnerability Scanner

Enter website URL (e.g., https://example.com)

Scan

Scan Results for: http://instagram.com

- HTTPS - Not Enabled
- Security Headers - All Present
- Directory Listing - Disabled (Safe)
- XSS - Not Detected
- SQL Injection - Not Detected

VulnQuest Pro - Vulnerability Scan Report

Generated on: 2026-01-16 19:25:03

Target	URL	Vulnerability	Severity
http://testphp.vulnweb.com	http://testphp.vulnweb.com	Missing Security Header	Low
Description: Content-Security-Policy header not set Solution: Configure your server to add: Content-Security-Policy			
http://testphp.vulnweb.com	http://testphp.vulnweb.com	Missing Security Header	Low
Description: X-Frame-Options header not set Solution: Configure your server to add: X-Frame-Options			
http://testphp.vulnweb.com	http://testphp.vulnweb.com	Missing Security Header	Low
Description: X-Content-Type-Options header not set Solution: Configure your server to add: X-Content-Type-Options			
http://testphp.vulnweb.com	http://testphp.vulnweb.com	Possible SQL Injection	Critical
Description: Database error message detected Solution: Use prepared statements, validate inputs, disable verbose database errors			
http://testphp.vulnweb.com	http://testphp.vulnweb.com	CSRF Risk	Medium
Description: Form found without CSRF protection Solution: Add CSRF tokens and validate them on the server			
http://testphp.vulnweb.com	http://testphp.vulnweb.com/admin	Exposed Sensitive Path	Medium
Description: Accessible path detected: /admin Solution: Restrict access using authentication, role-based access, or IP whitelisting			

